

Programação Aplicada

Interfaces Gráficos

Elementos Gráficos, AWT e Applets

Marco Veloso

marco.veloso@estgoh.ipc.pt

(Revisto e adaptado a partir do original de António José Mendes e Maria José Marcelino)

Agenda

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Packages, Code Conventions, Formatações e Administração

Debugging & Logging

Acesso a Bases de Dados relacionais através de JDBC

Sistemas Concorrentes (*Threads*)

Comunicação em Rede (*Sockets*)

Interfaces Gráficos

Agenda

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Elementos Gráficos, AWT e Applets

Sessão 0

Interface Gráficos

Componentes de um Interface Gráfico

Sessão 1

Gestão de Eventos

Contentores e Gestores de Posicionamento

Sessão 2

Interacção com o utilizador

Caixas de diálogo e menus

Sessão 3

Elementos Multimédia

Listagens e Tabelas

Formatações gráficas

Sessão 4

Agenda

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Elementos Gráficos, AWT e Applets

Interface Gráficos

Componentes de um Interface Gráfico

Gestão de Eventos

Contentores e Gestores de Posicionamento

Interacção com o utilizador

Caixas de diálogo e menus

Elementos Multimédia

Listagens e Tabelas

Formatações gráficas

Elementos Gráficos, *AWT* e *Applets*

Aplicação

Pode ser executada pelo sistema operativo **não necessitando** que qualquer outro programa a invoque (**autónoma**)

Applet

Não pode ser executada independentemente (**não autónoma**)

Tem que ser chamado a partir de HTML (uma página web ou de um “***Applet viewer***”)

Nesta primeira introdução aos gráficos vamos usar ***applets***

HTML

```
<TITLE>Applet title</TITLE>  
<APPLET CODE = Nome_do_Ficheiro.class  
    WIDTH    ="100"  
    HEIGHT   ="100"  
    ALIGN    ="MIDDLE"  
    HSPACE   ="0"  >  
</APPLET>
```

Ambientes de programação (IDEs) como o Eclipse **produzem automaticamente o código HTML necessário** a chamar a *applet*.

Integra ainda um “***Applet Viewer***” que permite visualizar a *applet* sem recorrer a um browser

Suporte para *Applets*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Java deixou de fornecer suporte para *Applets* e *Applet viewer* a partir da versão 11 do JSE

```
import java.applet.Applet;
```

```
public class Ex1TesteApplet extends Applet{  
    public void paint(Graphics g) {  
        g.drawRect(10,10,100,100);  
    }  
}
```



No entanto, a sua análise é relevante para o estudo de métodos gráficos no desenho e escrita de animações em painéis (**JPanel**) da **biblioteca AWT (*Abstract Window Toolkit*)**, analisados na secção seguinte

A execução de uma *applet* num JSE superior à versão 11 lançará a excepção:

```
java.lang.ClassNotFoundException: sun.applet.AppletViewer
```

```
Error: Could not find or load main class sun.applet.AppletViewer  
Caused by: java.lang.ClassNotFoundException: sun.applet.AppletViewer
```

Será necessário usar uma máquina virtual com versão anterior à 11 (e.g. JSE 1.8) ou **converter a *applet* num objecto **Panel** ou **Frame****

Ecrã gráfico

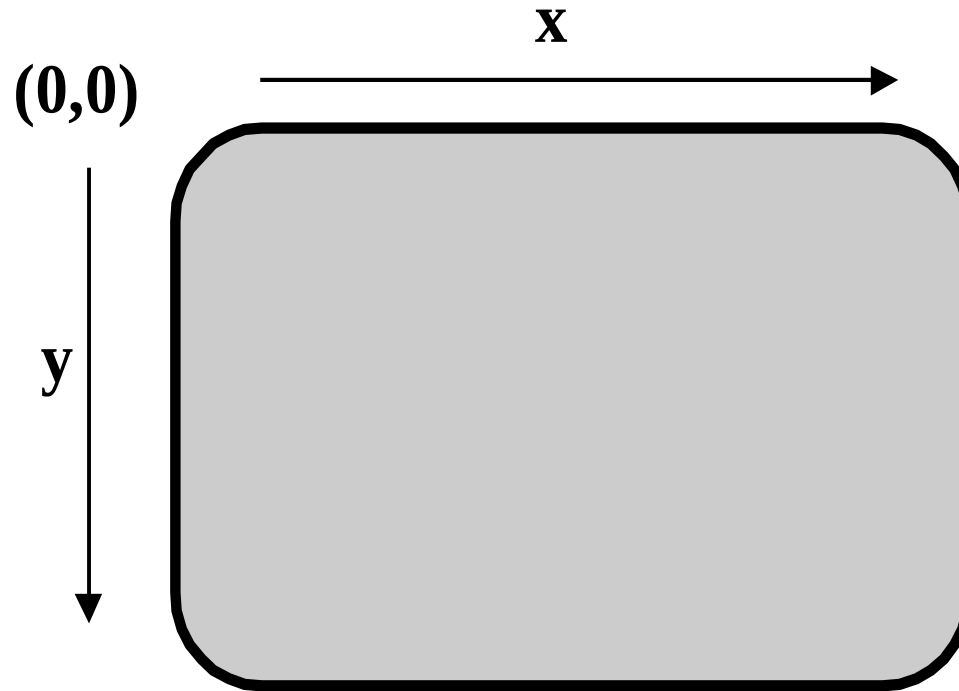
Os gráficos em computador são baseados em ***pixels*** (*Picture Element*)

Um *pixel* é **um ponto do ecrã** que pode ser apresentado numa qualquer cor (das possíveis no computador em causa)

O número de *pixels* disponível varia com a resolução da placa de vídeo do computador e do respectivo monitor

Resoluções típicas incluem 640x480, 800x600, 1024x768, 1152x864, 1280x1024 e 1980x1280

Coordenadas Gráficas



Exemplo de um gráfico em *Applet*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Um primeiro gráfico:

```
import java.awt.Graphics;  
import java.applet.Applet;  
  
public class Linha extends Applet {  
    public void paint (Graphics g) {  
        g.drawRect (0,0,100,100);  
    }  
}
```

Importar bibliotecas

paint é chamado automaticamente

Classe é baseada em *Applet*

Desenha um rectângulo iniciando no ponto (0,0) com 100 pixels de comprimento e 100 pixels de altura
g representa a área de desenho

Uma *Applet* não tem método `main()`, mas sim um método **paint()** responsável por desenhar ou actualizar os elementos gráficos no ecrã após uma alteração

O método **paint** devolve o objecto **Graphics** que representa a tela sobre a qual serão desenhados os elementos gráficos (e.g. **g.drawRect**), sendo igualmente invocado sempre que o ecrã necessita de ser actualizado

Exemplo de um gráfico em *Panel*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Uma vez que o código anterior (*Applet*) não é executável com JSE 11 ou superior, poderemos **adaptar o código para que recorra a um painel (*Panel*)** ao invés de uma *applet*:

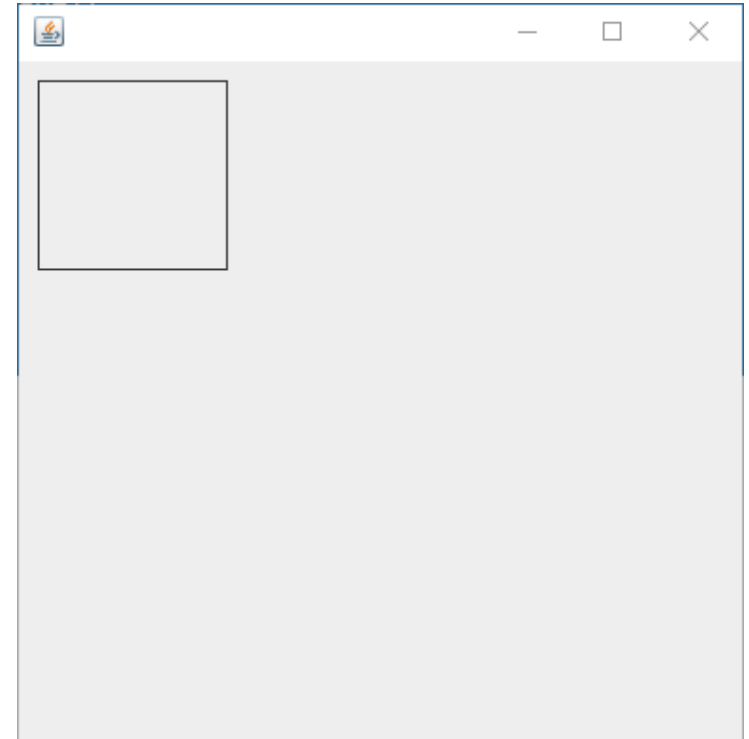
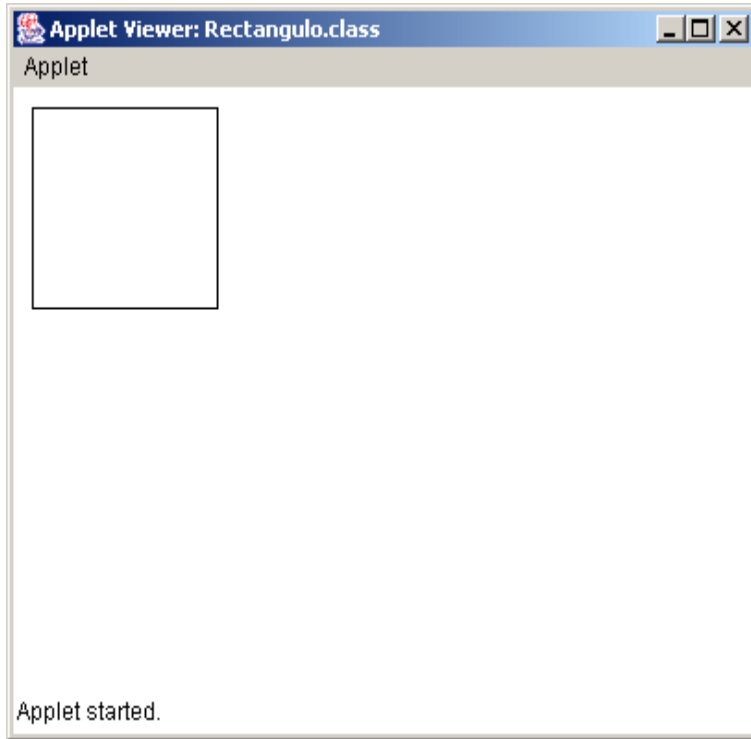
```
import java.awt.Graphics;  
import java.awt.Panel;  
  
public class Linha extends Panel{  
    public void paint(Graphics g) {  
        g.drawRect(10,10,100,100);  
    }  
}
```

Esta volta a ser uma aplicação autónoma, necessitando assim do `main()` para iniciar. É criada uma **Frame** (janela autónoma) onde é adicionado um painel (**Panel**), que permite obter o objecto **Graphics** para desenhar

```
import javax.swing.JFrame;  
  
public static void main(String [] args) {  
    JFrame app = new JFrame();  
    app.add(new Linha());  
    app.setSize(400,400);  
    app.setVisible(true);  
}
```

Exemplo de um gráfico em *Applet* e *Panel*

Interfaces Gráficas - Elementos Gráficos, AWT e *Applets*



Elementos Gráficos

Métodos Gráficos

Interfaces Gráficos - Elementos Gráficos, AWT e Applets

Método	Funcionalidade
<code>public abstract void drawArc (int x, int y, int width, int height, int startAngle, int arcAngle)</code>	desenha um segmento de arco. Os argumentos x e y especificam o canto superior esquerdo do rectângulo envolvente e width e height o seu comprimento e largura. Os argumentos startAngle e arcAngle especificam os ângulos de início e de tamanho do arco, respectivamente.
<code>public abstract void fillArc (int x, int y, int width, int height, int startAngle, int arcAngle)</code>	Idem, para um arco a cheio (interior preenchido).
<code>public abstract void drawOval (int x, int y, int width, int height)</code>	desenha uma oval inscrita no rectângulo de canto superior esquerdo no ponto x, y e comprimento width e largura height.
<code>public abstract void fillOval (int x, int y, int width, int height)</code>	Idem para uma oval a cheio .
<code>public abstract void drawLine (int x1,int y1, int x2, int y2)</code>	desenha um segmento de recta entre os pontos (x1,y1) e (x2, y2).
<code>public void drawRect (int x, int y, int width, int height)</code>	desenha um rectângulo com canto superior esquerdo no ponto (x, y) e comprimento width e largura height.
<code>public abstract void fillRect (int x, int y, int width, int height)</code>	Idem para um rectângulo a cheio .
<code>public abstract void clearRect (int x, int y, int width, int height)</code>	Apaga o rectângulo desenhado
<code>public abstract void drawString (String str, int x, int y)</code>	Escreve a string str na posição (x, y).
<code>public void setBackground(Color c)</code>	Coloca a cor de fundo a c.
<code>public abstract void setColor (Color c)</code>	Define a cor dos elementos a desenhar. Após a chamada deste método todos os elementos serão desenhados com a cor c.
<code>public abstract void setColor (int r, int g, int b)</code>	O mesmo que o método anterior, no entanto definido os valores RGB (0-255).

Os métodos gráficos são aplicados sobre o objecto **Graphics** que pode ser obtido através do método **paint(Graphics g)**, tanto em *applets* como em painéis (Panel):

```
g.drawRect(0, 0, 5, 5);
```

O objecto **Graphics** representa a tela (ecrã) sobre a qual serão **desenhados os vários elementos gráficos** (e.g. linhas, rectângulos, ovais, texto)

O método **paint()** é **invocado sempre que o ecrã for alterado** e é necessário redesenhar a tela

Também é possível obter o objecto **Graphics** solicitando directamente sobre o elemento gráfico que estamos a manipular através do método **getGraphics()**:

```
Panel painel = new Panel();  
Graphics g = painel.getGraphics();
```

Métodos Gráficos

Interfaces Gráficos - Elementos Gráficos, AWT e Applets

drawRect

```
g.drawRect (10,20,90,50);
```

(10,20)



```
fillRect(x, y, width, height);
```

(100,70)

```
drawRoundRect(x, y, w, h, arcWidth, arcHeight);
```

```
fillRoundRect(x, y, w, h, arcWidth, arcHeight);
```

```
clearRect();
```



Ângulo do canto

(10,10)

(10,10)

(25,50)

(50,25)

draw3DRect

```
g.draw3DRect (10,20,90,50,true) ;
```

```
fill3DRect(x, y, width, height, salient) ;
```

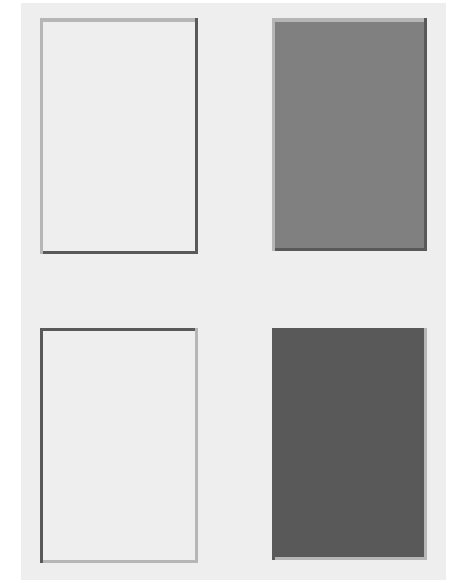
```
clear3DRect() ;
```

```
g.draw3DRect (25, 10, 50, 75, true);
```

```
g.draw3DRect (25, 110, 50, 75, false);
```

```
g.fill3DRect (100, 10, 50, 75, true);
```

```
g.fill3DRect (100, 110, 50, 75, false);
```



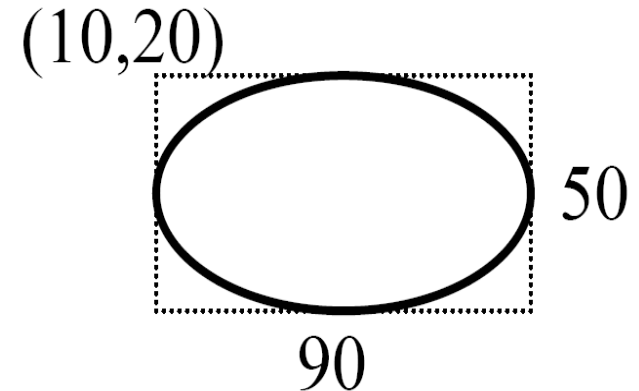
Nota: o uso de 3D apenas é visível quando a *applet* é integrada em Panels

Métodos Gráficos

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

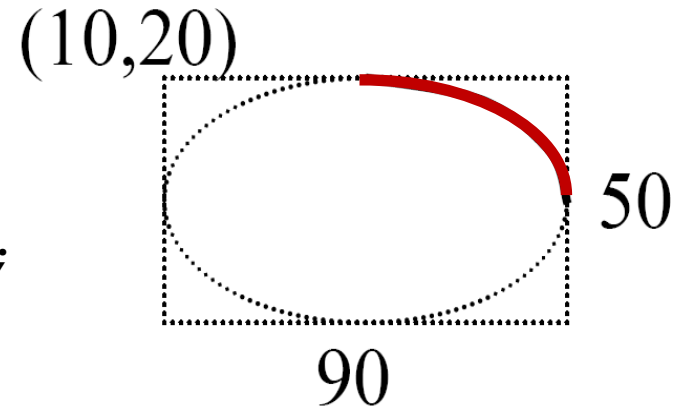
drawOval

```
g.drawOval (10,20,90,50);  
  
fillOval(x, y, width, height);  
  
clearOval();
```



drawArc

```
g.drawArc (10,20,90,50,0,90);  
  
fillArc(x, y, width, height,  
        start_angle, end_angle);  
  
clearArc();
```



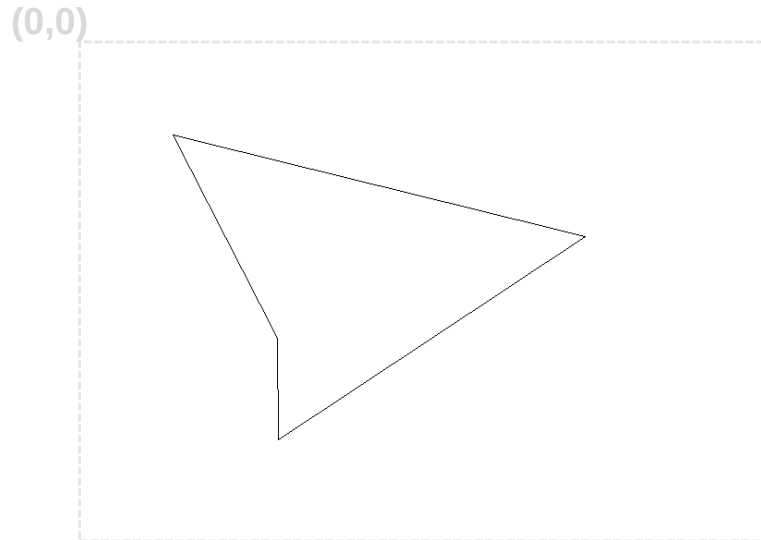
drawPolygon

```
drawPolygon(x, y, length);
```

```
clearPolygon();
```

Ex.

```
int [] x = {10, 23, 24, 56, 10};  
int [] y = {10, 30, 40, 20, 10};  
g.drawPolygon (x, y, x.length);
```



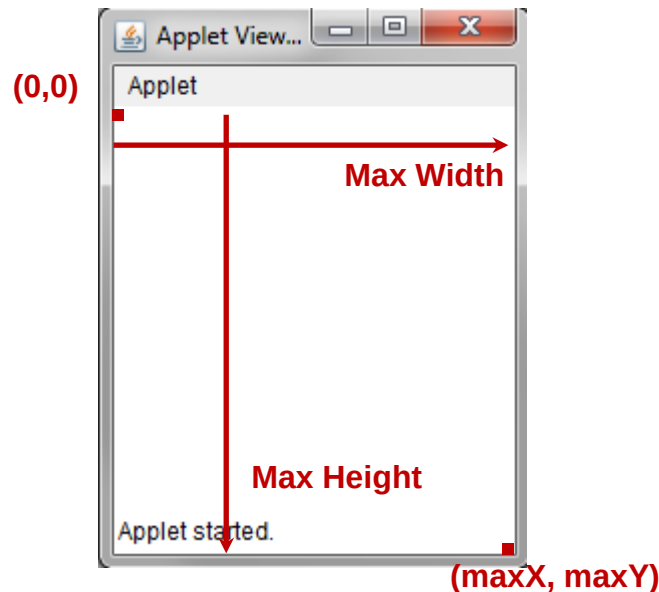
Dimensões de uma janela gráfica

Interfaces Gráficos - Elementos Gráficos, AWT e Applets

Obter as dimensões de uma componente gráfica
(*Panel*, *Applet*):

```
int maxX = getSize().width;
```

```
int maxY = getSize().height;
```



Definição de Cores

Interfaces Gráficos - Elementos Gráficos, AWT e Applets

Definição de cores (**setBackground**, **setColor**)

Definir a cor a partir de uma paleta de cores pré-definida:

*black, blue, cyan, darkGray, gray, green, lightGray,
magenta, orange, pink, red, white, yellow*

```
setColor ( Color.red ); // Há 13 cores pré-definidas
```

Definir a cor recorrendo ao método RGB:

```
Color c = new Color(R, G, B); // Red, Green, Blue
```

```
Color c1 = new Color(0, 255, 0); // verde
```

```
Color c2 = new Color(117, 23, 48); // castanho
```

```
setColor (c1); // usando um objecto do tipo Color
```

```
setColor ( new Color(255, 0, 0) ); // recorrendo ao método RGB
```

O método `getBackground()` devolve a cor de fundo do objecto

Mais alguns métodos gráficos

drawString

```
drawString(String, x, y);
```

Permite **escrever cadeias de caracteres na área gráfica**

```
EX. g.drawString ("Bom dia !", 10,20);
```

Escreve a frase “Bom dia !” a partir do ponto (10,20)

Usa a **fonte e o estilo que estiverem activos**

Definição de fontes

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

É possível **modificar os parâmetros da fonte** (classe **Font**) para obter um texto formatado pretendido.

Para o efeito é necessário **invocar um objecto de uma classe **Font****, definido na *package* **java.awt**, definindo três argumento: **nome da fonte**, **estilo da fonte** e **tamanho da fonte**, com a seguinte a sua sintaxe:

```
Font f = new Font(    "Nome_da_Fonte",  
                     Estilo_da_Fonte,  
                     Tamanho  
                     );
```

A seguir à declaração do objecto da classe **Font** é necessário utilizar um método denominado **setFont()**, para esta **ser aplicada ao objecto da classe **Graphics**** que estiver a ser utilizado

```
g.setFont(f);
```

Definição de fontes

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

O “**Nome da Fonte**” é uma *String* contendo o nome pelo qual a fonte é conhecida. Algumas das **fontes disponíveis** são as seguintes:

- “**TimesRoman**”,
- “**Courier**”,
- “**Terminal**”,
- “**Helvetica**”,
- “**Dialog**” e
- “**Σψμβολ**” (Symbol)

O “**Estilo da Fonte**” é uma variável que representa os tipos de **formatação de texto adicionais** que gostamos de dar aos nossos textos, por exemplo, realçando as palavras a **negrito** (*Bold*) ou a *Itálico*. A lista completa está definida na classe `Font`, mas os estilos mais usuais são o

- “**Font.PLAIN**”,
- “**Font.BOLD**”,
- “**Font.ITALIC**”

O “**Tamanho**” indica o número de pontos que devem representar a letra

Exemplo de definição de fontes

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

```
import java.awt.Graphics;
import java.awt.Font;
Import java.applet.Applet ;

public class Teste extends Applet {

    public void paint (Graphics g) {

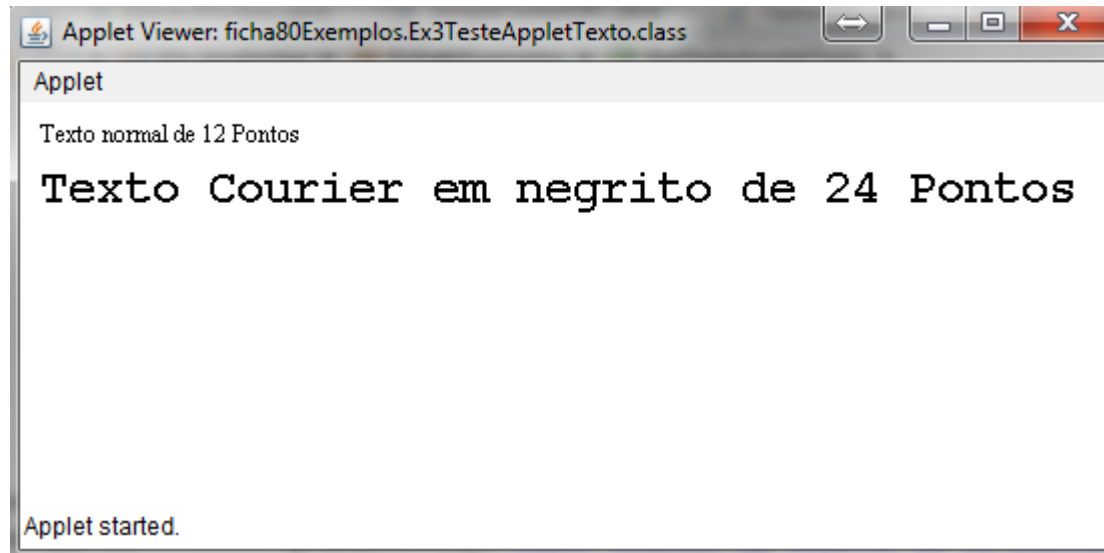
        Font f1 = new Font("TimesRoman", Font.PLAIN, 12);
        Font f2 = new Font("Courier", Font.BOLD, 24);

        g.setFont(f1);
        g.drawString("Texto normal de 12 Pontos", 10, 20);

        g.setFont(f2);
        g.drawString("Texto Courier em negrito de 24 Pontos",
                    10, 50);
    }
}
```

Exemplo de definição de fontes

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*



Impressão

Interfaces Gráficos - Elementos Gráficos, AWT e Applets

```
import java.awt.*;
import java.awt.print.*;

public class HelloWorldPrinter
    implements Printable { // Step 1

    private String textoToPrint = "Hello World!";

    public int print(Graphics g,
                    PageFormat pf,
                    int page)
        throws PrinterException {

        Graphics2D g2d = (Graphics2D) g;
        // conversão do referencial (X,Y) para o
        // formato da folha a imprimir, definindo
        // a posição do ponto (0,0)
        g2d.translate(pf.getImageableX(),
                    pf.getImageableY());

        // Definição do conteúdo a imprimir
        g.drawString(textoToPrint, 100, 100);

        if (page > 0) {
            // só possuímos uma página
            return NO_SUCH_PAGE;
        } else
            return PAGE_EXISTS;
        }
}
```

```
public void printPage() {

    // Step 2
    PrinterJob job = PrinterJob.getPrinterJob();

    // Step 3
    job.setPrintable(this);
    //job.setPrintable(new HelloWorldPrinter());

    // Step 4
    boolean ok = job.printDialog();
    if (ok) {
        try {

            // Step 5
            job.print();
        } catch (PrinterException ex) {
            // Falha na impressão
        }
    }
}

public static void main(String args[]) {
    HelloWorldPrinter hello =
        new HelloWorldPrinter();
    hello.printPage();
}

} // end of class
```

Dimensões de uma Applet

Interfaces Gráficos - Elementos Gráficos, AWT e Applets

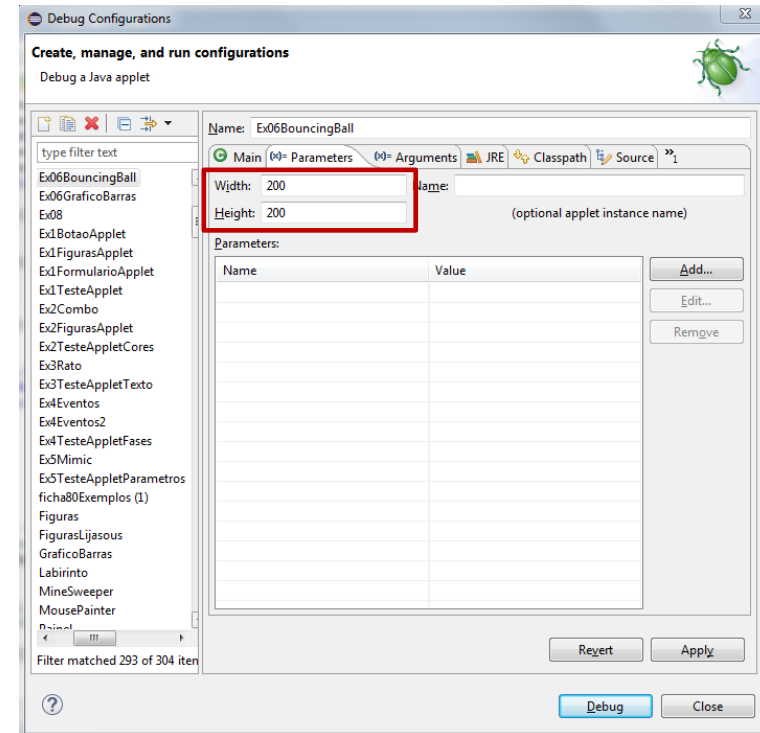
Sendo uma *applet* integrada noutra aplicação (*applet viewer*), o seu **tamanho é definido pelo respectivo applet viewer**

Por exemplo, no IDE Eclipse, é possível definir o tamanho de cada *applet* através das configurações de execução:

run > run configurations > parameters

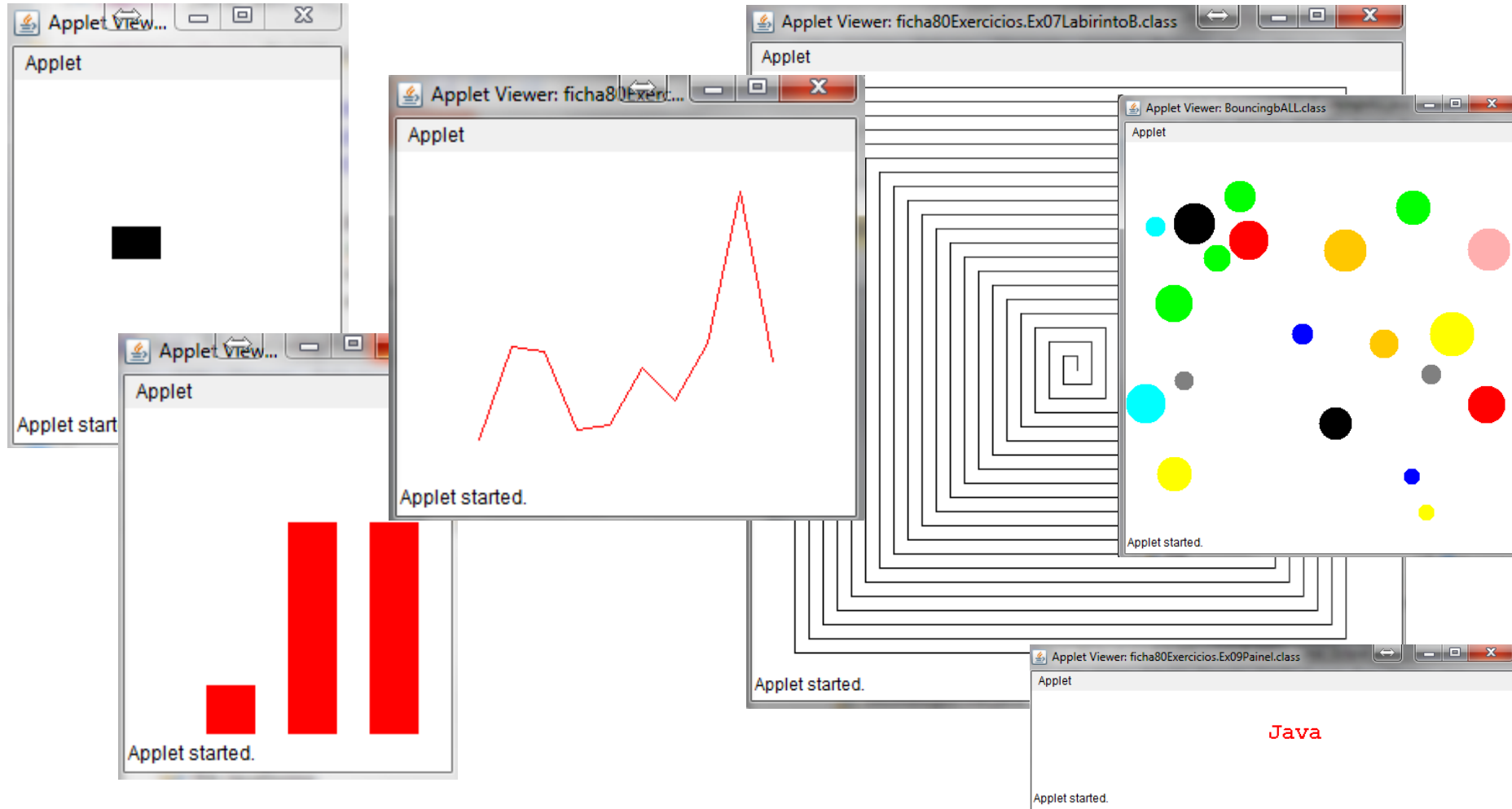
alterando os valores dos campos **Width** e **Height**, que definem o comprimento e largura inicial da *applet*. Este procedimento **passa os valores como parâmetros** para a *applet* durante a execução

Porém, é possível forçar o tamanho inicial da *applet* por código, através da invocação do método **setSize(comp, larg)**. Este método deve ser invocado através do método de arranque das *applets* (**init()**)



Exemplo de animações gráficas

Interfaces Gráficas - Elementos Gráficos, AWT e Applets



Fases de uma *Applet*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Fases de uma *Applet*

Fases de uma *Applet*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Não existindo um método `main()`, existem na *Applet* um conjunto de métodos que representam as fases mais importantes da vida de uma *Applet*:

Inicialização – Esta é a fase em que a *Applet* é pela primeira vez carregada. Nesta fase podem ser realizadas acções, tais como, a criação de objectos, inicialização de variáveis, etc., invocada antes do método `paint()`. O método correspondente a este estado é o `init()`. Uma *Applet* só passa por esta fase uma vez

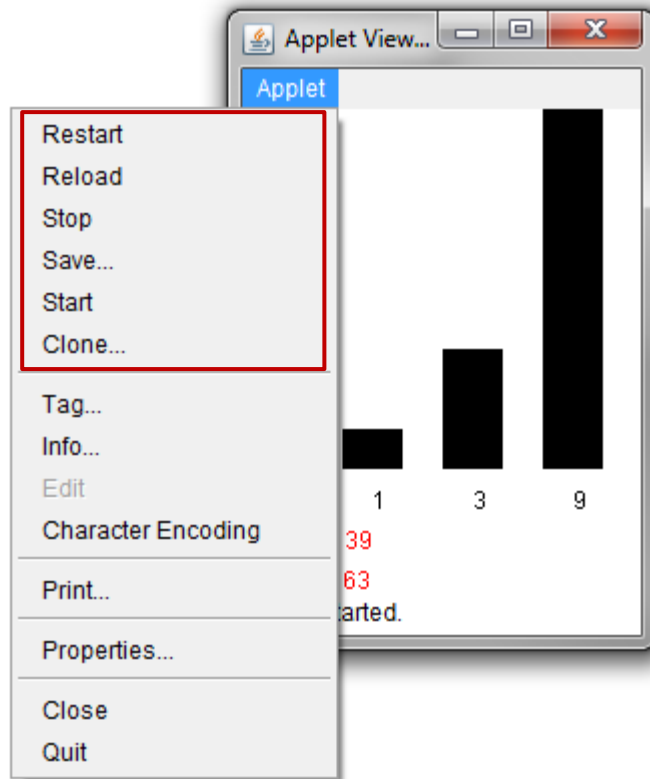
Começar – Esta fase ocorre a seguir à inicialização, ou então, depois de a *Applet* ter sofrido uma paragem. Ao contrário da fase anterior, uma *Applet* pode passar várias vezes pela fase “Começar”. O método correspondente é o `start()`

Parar – Esta fase implica uma paragem na execução da *Applet*. É muitas vezes utilizada em conjunto com o estado “Começar”, para certos tipo de comportamento de *Applets*. O método correspondente é o `stop()`

Destruir – Este é o estado terminal de uma *Applet*. O método associado é o `destroy()`

Fases de uma Applet

Interfaces Gráficas - Elementos Gráficos, AWT e Applets



Fases de uma *Applet*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

Estes quatro métodos estão definidos na classe **Applet**, entre muitos outros

Contudo, se for definida uma *Applet* vazia, o seu comportamento também será nulo, uma vez que nenhum dos métodos executa alguma acção

Para **criar comportamentos é necessário reescrever os métodos (*Override*)**. Para isso basta declarar os métodos no interior do código das *Applets*, tendo o cuidado de os **declarar como *public***

Por exemplo se pretendermos definir um **comportamento inicial** para uma determinada *applet*, seria necessário fazer a **declaração do método `init()`**:

```
public void init() {  
    // código de definição do comportamento  
    setSize(300, 100);  
}
```

O comportamento de uma *applet* é, normalmente, **definido à custa da reescrita dos métodos `init()`, `start()` e `stop()`**

Não é habitual associar comportamentos ao método **`destroy()`**

Fases de uma *Applet*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

O código destes métodos só será executado quando a *Applet* estiver no estado correspondente

Em todos os casos em que tenhamos **desenhos ou operações gráficas sobre a *Applet***, iremos recorrer a um método designado por **paint()**, invocado sempre que se pretenda enviar algo para o ecrã

Este será **utilizado várias vezes durante a execução de uma *applet***, por exemplo, para gerar uma animação (repetidos envios de imagens para uma janela) ou mesmo para **regenerar uma imagem que foi tapada por uma janela**

O **redesenho da *applet*** pode ser invocado deliberadamente através do método **repaint()**, o que fornece as bases para uma animação simples

Passagem de parâmetros às *Applets*

Interfaces Gráficas - Elementos Gráficos, AWT e *Applets*

É possível passar parâmetros para o interior de uma *Applets*. Estes parâmetros são identificados por um nome e por um valor. A sintaxe a utilizar é a seguinte:

```
<APPLET CODE = "%S"  
        NAME = "%S" WIDTH = 400 HEIGHT = 300 ALIGN = middle>  
  <param var1 value = "valor1">  
  <param var2 value = "valor2">  
  (...)  
  <param varN value = "valor3N">  
</APPLET >
```

O valor passado pode ser recuperado pelo programada da *Applet*, no interior do método `init()`, sendo para tal chamado um método denominado `getParameter()`, que aceita como argumento o nome da variável a passar (sempre uma *String*)

No caso de ser pretendido passar um valor de outro tipo que não uma *String*, a conversão tem que ser feita depois de obtida a *String* com o método `getParameter()`

Passagem de parâmetros às *Applets*

Interfaces Gráficas - Elementos Gráficos, AWT e *Applets*

Este exemplo serve para ilustrar a **passagem de parâmetros** para uma *Applet* Java

```
import java.awt.Graphics
import java.applet.Applet;

public class Teste extends Applet {
    String frase;

    public void init() {
        frase = getParameter("var1");
    }

    public void paint(Graphics g) {
        g.drawString(frase, 10, 25);
    }
}
```

A *String* a enviar para o ecrã é passada por intermédia da variável “frase”, que toma o valor do parâmetro de nome “var1” por execução do método **getParameter()**

Passagem de parâmetros às *Applets*

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

O código HTML correspondente, onde se faz a **passagem do referido parâmetro**, é o seguinte:

```
<HTML>
  <HEAD>
    <TITLE>Applet Test Page</TITLE>
  </HEAD>
  <BODY>
    <APPLET CODE = "Teste.class"
      WIDTH = 400 HEIGHT = 300 ALIGN = middle>
      <PARAM
        NAME = "var1"
        VALUE = "A minha applet modificada!">
      </APPLET>
    </BODY>
  </HTML>
```

O texto desenhado pela *Applet* tem, neste caso, a particularidade especial de ter sido passado para o seu interior, através do código HTML.

Para o utilizador final o efeito é o mesmo, mas a utilização de parâmetros passados através de HTML vem trazer mais flexibilidade, ao **permitir o ajuste dos parâmetros directamente pelo utilizador**, sem necessidade de conhecimento da *Applet* e sem compilação.

Referências

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

“Programação Orientada a Objectos”

António José Mendes

Departamento de Engenharia Informática, Universidade de Coimbra

“Java Swing”, 2ª Edição

James Elliott, Robert Eckstein (Editor), Marc Loy, David Wood, Brian Cole

O'Reilly, ISBN: 0596004087

“Thinking in Java”, 4ª Edição, Capítulo 14 “Creating Windows & Applets”

Bruce Eckel

Prentice Hall, ISBN: 0131872486

“The Java Tutorial – Creating a GUI with JFC/Swing”

Java Sun Microsystems

<http://java.sun.com/docs/books/tutorial/uiswing/index.html>

Referências

Interfaces Gráficos - Elementos Gráficos, AWT e *Applets*

"The Definitive Guide to Java Swing", 3ª Edição

John Zukowski

Apress, ISBN: 1590594479

"Fundamentos de Programação em Java 2", Capítulo 12 *"Interfaces Gráficos"*

António José Mendes, Maria José Marcelino

FCA, ISBN: 9727224237

Revisto e adaptado a partir do original de António José Mendes e Maria José Marcelino